Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime

Alvaro Rivera

2026-05-18

Abstract

This paper is a design theory contribution in the sense of Hevner, March, Park, and Ram (2004): it introduces a structural construct — *program-value separability* — derives the runtime consequences that follow from it, and presents one realization in production as confirmation that the construct is realizable. Compilation of a domain-specific language program to native code, caching of compiled programs across invocations, and dense journaling of operations rather than their effects are commonly treated as independent runtime optimizations. This paper argues that they are not — that all three are downstream consequences of a single structural property of DSL programs, which we name *program–value separability*: the syntactically decidable condition that a program declares its parameters explicitly rather than embedding values in its body. We trace four runtime faces (compilation, caching, dense journaling, replication-bounded entropy) as projections of this single precondition, characterize one concrete realization in a CQRS + Actor + Event Sourcing runtime — the same Puppeteer framework introduced in the prior paper of this series — and report empirical magnitudes from measurements over two independent open-source DDD aggregates (dotnet/eShop's Order and kgrzybek/modular-monolith's SubscriptionPayment): a compiled- versus-interpreted speedup that scales with DSL-bound work (1.49× to 4.10×); a cold compile cost amortized within hundreds of invocations; a journal storing 99.8% of parametric workloads as compact action references, 3.5× to 5.6× denser than the equivalent literal-script storage. The principle generalizes beyond any one runtime: program-value separability characterizes the structural condition any DSL runtime must satisfy to admit compilation, caching, and dense persistence at all.

Program–value separability

TL;DR

Compilation, caching, and dense journaling are **not** features layered atop interpretation. They **are** downstream consequences of a single structural commitment: values live outside the script, not embedded within it.

The transition often described as "adding compilation" inverts the causal order. A DSL whose programs embed their values has no stable identifier, no template to cache, no separable sub-program to compile ahead of time — interpretation is not a stylistic choice but a structural necessity. Once values move outside the script, the program becomes a function awaiting its arguments — *F(x , x , …, x)*, as the runtime documents itself.

From that single act of separation, four runtime decisions cohere. Scripts with externalized parameters are eligible for compilation to IL via Expression trees, cache as reusable programs with action identifiers, persist to the journal as compact action entries, and replicate with entropy bounded by argument vectors. Scripts with hardcoded values are not cached, run once, and persist as their literal text. The four faces are not parallel design choices; they are projections of a single precondition.

This precondition operates on the substrate characterized in the previous paper of this series — Puppeteer's journal as a homoiconic representation, code and data sharing one form. We adopt the colloquial label *code-as-data* from this point on; the formalism stands as established. The compactness of these entries is not merely syntactic: each names an operation substantially richer than its surface signature, with the asymmetry scaling with the domain library's depth (§5.5).

This is the principle of program–value separability: a DSL program becomes identifiable, compilable, cacheable, and persistable densely only when it is separable from its values. Externalized parameters are the structural precondition under which compilation, caching, and dense journaling become possible at all.

Claims this paper makes

1. **Causal inversion: program–value separability is a necessary condition.** Compilation, caching, and dense journaling in the runtime described here are not optimizations layered atop interpretation. They are

downstream consequences of *program–value separability* — the structural property that values live outside the script body, not embedded within it. A DSL program embedding its values has no stable identifier, no template to cache, no separable sub-program to compile ahead of time. Separability is necessary, not optional. *(Verification: logical / structural — a script embedding its values produces a different cache key per invocation, making caching structurally pointless. The runtime documents itself: "Un script funciona como F(x1,x2,…,xn)" (`ActorHandler.cs:1085`) — F is the program; the x are its externalized values.)*

2. **DSL programs admit a structural binary taxonomy.** *Parametric scripts* are separable from their values: cacheable as reusable programs with action identifiers, persistable as compact action entries in the journal, and economically compilable. *Literal scripts* are non-separable: never cached, ephemeral, and persisted as their literal text. Whether either class is compiled or interpreted at runtime is governed by a separate per-actor policy that, by default, follows the parametric/literal split. *(Verification: ActorHandler.cs:1091-1093 documents the rule verbatim. Implementation: PrepareCommandProgram (ActorHandler.cs:1097-1148) decides via parameters.HasUserParameter() and assigns one of three JournalEntry values — IsScript (literal), IsNewAction (parametric, first invocation), IsExistingAction (parametric, cached invocation). Compilation flag set per program by AdjustCompilationMode (Program.cs:134-150) under the CompilationModePolicy enum.)*

3. **Interpretation and compilation are strategies over a shared AST substrate.** They are not separate execution paths over translated representations. Both walk the same AST: interpretation walks it directly; compilation specializes it into IL via Expression trees. Both paths pay the same parsing cost; only the compiled path pays the compilation cost. *(Verification: Program.Execute() (interpretation) and Program.ExecuteExpression() → ProgramExpression().Compile() (compilation) operate over the same Statement tree produced by Parser.Parse().)*

4. **Journal density emerges from separability, not from a compression scheme.** A stable F(x , …, x) collapses repeated invocations into (actionId, values) tuples, with the script definition persisted once and referenced thereafter. The compactness is structural, not designed. *(Verification: Diary.WriteNewActionEntry (definition + first invocation values) vs Diary.WriteActionEntry (actionId + values only on subsequent invocations).)*

5. **Verb richness: surface vs depth.** A single DSL verb may invoke orchestrations richer than its surface signature suggests. This is not the result of translation between layers; the DSL is the invocation language for domain operations implemented directly in the host language. Journal density is therefore the asymmetry between small persisted tokens and the live-domain operations they represent. *(Verification: §4 develops the*

construct and §5.5 measures it on two open-source DDD aggregates — eShop's `Order` purchase verb dispatches 9 host-language invocations with a 24-method static closure (/ 2.7×); Grzybek's `SubscriptionPayment` verb dispatches 2 with a 73-method closure (/ 36×). The magnitude of the asymmetry is a function of how much the host's persistence anchor compresses its domain, not of the runtime mechanism.)

6. **Hot-loaded DSL programs over a stably-loaded domain.** The runtime supports hot-loaded DSL programs against a stably-loaded domain library: the assemblies configured as the actor's domain libraries are reflectively cached at first use; new DSL scripts can combine the types they carry in new ways without assembly reload. This enables ad-hoc procedure invocations against a live, stateful actor — without redeploying endpoints or altering the domain library. *(Verification: DomainLibraries.GetOrLoad(params Assembly[]) caches public types statically per deduplicated assembly set; ActorV2.Using(scriptForChk, scriptForCmd) introduces new scripts per invocation against that cached surface area.)*

When NOT to use this approach

The principle of program–value separability has limits, and the implementation pattern that realizes it has narrower limits still. We name six regimes in which the analysis presented here either does not apply, applies only partially, or invites overreach.

A. This paper is not advocating for compiling all DSL programs unconditionally

The structural taxonomy described here distinguishes parametric from literal scripts; both are first-class. Literal scripts — ad-hoc oracle invocations against a live actor, exploratory queries against a stateful runtime, configuration-time experiments — pay zero amortization cost because they incur no compilation. Treating them as legacy or transient would discard a structurally valid mode of interaction.

B. This paper does not address consistency under concurrent actors

The runtime described operates with strict per-actor ordering and isolation. Multi-actor consistency, reaction propagation across actor boundaries, and the contract under which observable state remains coherent are treated formally in a companion paper of this series.

C. This paper does not claim program–value separability is sufficient for the four-fold coherence

Separability is necessary but not sufficient. SQL prepared statements achieve compilation and caching while exhibiting only two of the four faces — the per-

sisted artifact is row data, not the parameterized statement itself, so homoiconic persistence does not emerge. Other realizations may exhibit further subsets. Puppeteer demonstrates one realization where all four faces emerge by structural commitment; the converse is not entailed by separability alone.

D. This paper does not extend separability to general-purpose programming languages

The principle is formulated for DSL runtimes — domain-specific languages whose programs are short, structured, and amenable to identification by their textual form. General-purpose programming languages introduce variables captured from enclosing scope, side-effecting state in the host, and identity that exceeds the parameter contract. Whether separability admits a meaningful formulation in that broader setting is a separate question, and we do not take it up here.

E. This paper does not claim that hot-loaded DSL programs replace traditional deployment

Claim 6 articulates a runtime capability, not a deployment recommendation. Hot-loaded DSL programs against a stably-loaded domain enable ad-hoc invocation; redeployment of the domain library remains the appropriate move when the implementation itself changes. The two are complementary, not alternatives.

F. This paper does not address security, sandboxing, or resource bounds for hot-loaded scripts

Claim 6 grants a running actor the ability to accept and execute DSL programs formed at call time. The structural argument made here is silent on the operational concerns that capability raises in production: sandboxing of the DSL evaluator, per-script resource limits (CPU, memory, wall-clock), authentication and authorization over which callers may submit ad-hoc invocations, and the trust model under which the supplied script is admitted at all. These concerns require separate machinery — out of scope for the conceptual contribution offered here, but unavoidable for any deployment that exposes hot-loaded invocation beyond a closed boundary.

---

1. Introduction

This paper makes a design theory contribution. It identifies a structural property of DSL programs that prior literature has touched but not isolated as one — the construct: *program-value separability*; derives the runtime consequences that follow when the property is held — the principles: compilation, caching, dense journaling, replication-bounded entropy; and presents an instantiation

— a system in which those principles have been realized in production — as confirmation that the construct is realizable. The contribution is conceptual; the instantiation is the existence proof of realizability, not the substance of the claim. The genre is the one Hevner, March, Park, and Ram (2004) name design science research: empirical evidence is presented in the form of a working artifact rather than a controlled experiment.

1.1 Calibrating the surface

You already understand this taxonomy. A SQL prepared statement is parameter-separable: the database parses it once, builds a query plan once, and reuses both for thousands of invocations with different bind values. An ad-hoc query offers no such surface — each is a fresh string, parsed and planned afresh. The same structural distinction governs DSL programs in any runtime that aspires to compilation, caching, and dense persistence. What changes between SQL and the runtime described here is not the principle, but the surface to which it applies.

That surface tends to be unfamiliar in a specific way. The professional reader of this paper has very likely spent a career persisting application state in relational tables. Where event sourcing has appeared in their work, it has typically appeared in tabular form: events as rows, schemas as projections, queries as joins against materialized views. The implicit assumption — that every representation of state ultimately resolves into something table-shaped — is not a failure of imagination but the horizon that industrial practice has produced over several decades. A runtime in which the persisted artifact is the program itself, in which density emerges from separability rather than from compression of rows, has had no canonical place in that landscape.

Calibration begins by separating two layers that the dominant paradigm tends to collapse into one. The first layer is the implementation: domain types, methods, business rules — written in the host language, indistinguishable in form from any well-designed object-oriented or functional library. The second layer is the invocation: the language by which a running actor is commanded and queried. In runtimes of the kind described here — of which Puppeteer, the framework introduced in the prior paper of this series, is one — that invocation language is a small DSL whose programs read as scripts, persist as journal entries, and execute as either compiled lambdas or interpreted walks over their own AST. The two layers are not translations of one another. The host language carries the implementation; the DSL carries the contract by which the implementation is reached. Conflating them — assuming the DSL must duplicate, transcribe, or shadow the host code — is where most readings of the architecture begin to diverge from its actual structure.

Three readings should be pre-empted at the outset: this is not double programming, not a translation layer, not a mapping specification. The domain implementation lives once, in the host language. The DSL is the language by which

that implementation is invoked. The relationship is closer to that between SQL and a relational engine than to that between a model and a generated DTO: SQL does not duplicate the engine's logic; it names operations that the engine performs. A short SQL statement can trigger joins, locks, and execution plans far heavier than its surface text. The DSL of an actor runtime carries the same kind of leverage — small surface, deep behavior — and the persisted journal records not the unfolding of that behavior but the invocations that produced it. With the surface calibrated, the formal question can now be asked: what does it take, structurally, for a DSL program to be compilable, cacheable, and amenable to dense persistence?

1.2 A formal characterization

The question can be sharpened. Compilation requires a stable artifact to specialize. Caching requires a stable identifier under which to file the result of that specialization. Dense journaling — persistence that does not balloon with the size of every invocation — requires a stable description of the operation that can be referenced rather than copied. Each of these three demands the same property of the program: that it admit, at the moment it is named, a separation between the program itself and the values it will receive.

This property is **Program–Value Separability**. A DSL program is separable when its textual form names an operation parameterized by values supplied externally at invocation, rather than carrying those values within its body. Separability is structural, not stylistic: it is a property the program either has or does not have. It is decidable syntactically at preparation time, by inspection of the parameter declarations on the program's surface. The principle that follows admits a compact statement: a necessary condition for cacheable specialization, stable-key caching, and dense journaling of any DSL program is program–value separability. Necessity is the strong claim; the limits of what separability does not entail are taken up shortly.

Separability admits a binary structural taxonomy of DSL programs. A **parametric script** is separable from its values: it is filed under a stable identifier in a runtime cache and persisted to the journal as a compact reference plus an argument vector. A **literal script** is non-separable: it is not cached, runs once as written, and persists as its literal text. Whether either class is compiled or interpreted at runtime is governed by a separate per-actor policy that, by default, follows the parametric/literal split — but admits override. Under that default policy, the runtime documents the joint regime in its own comments: scripts without user parameters are interpreted, uncached, and persisted as Script entries; scripts with user parameters are compiled, cached with an ActionId, and persisted as Action entries. The two strict properties — caching and journal entry format — are encoded as values of a single enum decided at the moment a program enters preparation. The taxonomy is not analytical but operational: the runtime acts on it.

Necessity is not sufficiency. A program that admits separability gains the structural possibility of being cached, persisted as a compact reference, and amortizing its compilation — but does not by that fact alone exhibit dense journaling. SQL prepared statements illustrate the gap: they are separable, they cache, they compile — yet the persisted artifact in the database is row data, not the parameterized statement itself. The third face — a journal in which the named operation is the persisted entry, not its downstream effects — does not emerge from separability alone. It requires the further commitment, code-as-data, that the runtime treat its programs as the persisted artifact: what is recorded in the journal is the program parameterized and the values it received, not the state changes those values produced. The sections that follow trace how separability, joined to code-as-data, yields the three faces in turn — and where a runtime that holds separability without the second commitment falls short.

The DSL program characterized above — the textual artifact with declared parameters — is the unit of separability. When the term *program* appears at the substrate level in subsequent papers of this series (the unit that is recorded, replicated between nodes, and replayed against state), it denotes the pair: a **domain library** (the compiled classes and verbs against which the DSL is interpreted, loaded statically and identical across replicas) and a **journal of invocations** (the ordered sequence of action references with parameter bindings that the runtime has applied). Replay is the deterministic re-execution of the journal against the library; what persists, replicates, and reconstructs state is this pair. A DSL program in the sense of §1.2 is the unit; the substrate-level program is the cumulative state-producing artifact built from many DSL invocations executed against a stable library. The two readings are not in tension: separability of the unit is what makes the cumulative artifact dense; the cumulative artifact is what makes the unit's separability operationally consequential.

2. Consequences of separability

2.1 Compilation as specialization

The first consequence of separability is that the program admits ahead-of-time specialization. A separable program is, in form, a function awaiting its arguments — its body refers to values that will arrive at invocation, not to literals fixed at write time. That structural shape is precisely what a specializer requires. Given the program once, the runtime can resolve its references, lower its abstract syntax tree into a typed expression tree, and emit the executable form in advance of any specific call. The values are bound later; the work of preparing the program is done once.

The pipeline that produces this specialized form proceeds in three stages. The parser yields an abstract syntax tree in which user parameters appear not as values but as named slots — placeholders to be supplied at invocation. The runtime then traverses that tree and emits a typed expression: each named slot is bound to a parameter expression of the host runtime, and each operation

of the script is bound to the corresponding host-language method or property access. Compilation of that expression yields an executable delegate over the parameter list — a function that takes the values supplied at invocation and runs the program against them.

Specialization, considered alone, is a one-time cost paid against an indefinite number of invocations. The parser runs once for the program; the expression tree is constructed once; the compiler emits the delegate once. Each subsequent invocation supplies new values into the existing delegate and runs through host-language code that has already been resolved, typed, and emitted. The amortization is structural: a separable program admits compilation that pays for itself across repeated invocation, while a non-separable program offers nothing to amortize against. The work, however, is only economical if the specialized form persists between invocations.

2.2 Caching as identification

Persistence between invocations requires an identifier under which to file the work that has been done. Separability supplies that identifier directly: the script string of a separable program is stable across invocations because its values are external to it. The same parameterized program text, encountered a second time, is recognizable as the same program — and the cached specialization belongs to it. A non-separable program has no such anchor; every invocation produces a different string, and no two invocations are the same program at all. Caching is not an addition; it is what becomes possible once the program has a stable name.

The mechanism is direct. On first encounter, the runtime parses the script, prepares its specialized form, and stores both under an action identifier — a stable handle assigned to the script when it is first encountered. The script string serves as the lookup key; the action identifier serves as the persistent reference under which the program's specialization lives. On subsequent encounters with the same script, the lookup succeeds, and the runtime retrieves the specialization without repeating the work that produced it. The cache is not a memoization of values; it is a registry of programs.

On a cache hit, the work that has been preserved is the program itself: the resolved tree, the typed expression, the compiled delegate. The values arriving at this invocation are bound into the delegate's parameter slots and the program runs against them. From the program's standpoint, nothing has changed between the first invocation and the thousandth — the script names the same operation, parameterized by the same slots, executed against the same domain. What differs across invocations is only the values supplied; what persists is the program.

2.3 Dense journaling as reference

The third consequence of separability is that persistence becomes reference rather than duplication. The program, having earned a stable identifier in the cache, can be written to the journal once — as a definition, with the script's text and its parameter declarations recorded in full. Each subsequent invocation is recorded not by duplicating the program text but by referring back to that definition under its identifier, accompanied only by the values supplied at that invocation. Where a non-separable runtime must record every invocation as a complete script — body, values, and all — a separable runtime records the program once and its invocations as compact references against it.

The mechanism distinguishes three kinds of journal entries. When a parametric program is encountered for the first time, the runtime writes a definition entry: the script's text, its parameter declarations, and the values supplied at this first invocation, all recorded together under the program's newly assigned identifier. Subsequent invocations of the same program write only a reference entry — the identifier of the definition, plus the values for that call. A non-separable program, having no stable identifier, is written each time as a script entry: its text in full, since it cannot be referred back to anything that came before. Three entry kinds, decided mechanically at preparation time from the program's separability, encode the program's relation to its identity.

The density follows. For a parametric program invoked many times, the journal's storage scales with the cumulative size of the argument vectors, not with the cumulative size of the script's body — the body has been recorded once, and every later invocation costs only the bytes of its values. This is what makes the journal dense: nothing is copied that has been preserved by reference, and what survives is the operation that took place, not the state that resulted. The persisted artifact is the program — the script as text, parameterized, with its invocation values — rather than the downstream effects of running it. In code-as-data terms, the program lives in the journal as itself, not as a record of what it produced. Compilation, caching, and dense journaling are thus not independent optimizations but successive consequences of the same structural property: once a program is separable from its values, it becomes nameable; once nameable, it becomes cacheable; once cacheable, it becomes referable in persistence. The mechanism by which the runtime accomplishes this — the pipeline that compiles, the cache that retains, the journal that references — is the subject of §3.

3. Realization in a concrete runtime

3.0 Origins of the instantiation

The realization described in this section was not engineered to demonstrate the principle articulated above. The principle was identified by inspecting a runtime that had, over years of independent development for production use, come to satisfy it. The genealogy is structural rather than programmatic — first the

artifact, then the principle that explains why the artifact cohered. What follows describes the runtime as it stands; the relationship between its mechanisms and the principle of §1.2 is a recognition, not a derivation.

3.1 Compilation pipeline

The mechanisms described here are not independent engineering decisions but direct realizations of the separability principle established in the preceding sections. In the runtime described, the compilation pipeline runs from text to executable in three well-defined stages. A `Parser.Parse()` call lowers the script into an abstract syntax tree of `Statement` and `AstExpression` nodes. Each `Statement` carries an `ExecuteExpression` method that, when traversed, emits a `System.Linq.Expressions` node bound to the host parameters and the host-language operations the script names. The runtime composes these into a single `Expression.Lambda` and calls `.Compile()` to produce an executable delegate. Three stages — parsing, lowering, and compilation — produce the artifacts on which the runtime operates: the AST, the expression tree, and the compiled delegate.

The mechanism is straightforward to follow in the source. `PrepareCommandProgram` (`ActorHandler.cs:1097-1148`) orchestrates the first encounter: it rents a parser from a pool, parses the script, and decides — by parameter presence — whether to enter the parametric path. On the parametric path, the program's `ProgramExpression` method (`Program.cs:182-244`) walks the AST: for each Statement, it calls the Statement's `ExecuteExpression`, which returns an `Expression` node bound to the runtime's parameter and output expressions. These nodes accumulate into a `BlockExpression`, which `ProgramExpression` wraps in `Expression.Lambda<Func<Parameters, Output, string>>` (`Program.cs:244`). The compilation itself is one line: `_executable = programExpression.Compile()` (`Program.cs:163`). The compiled delegate lives in the program's `_executable` field; subsequent invocations skip the compile step and call the delegate directly with the values supplied at the call site.

The same mechanism applies one level deeper, to a feature the runtime calls *Eval parameters*. A parameter declared with the `Eval` modifier carries its own sub-script, which the runtime treats as a separable program in miniature: parsed once, compiled to its own delegate, and cached against the parameter's name. The cache structure is a dictionary keyed by parameter name to a tuple of the eval script and its compiled executable (`Program.cs:305`). On each invocation, the runtime compares the parameter's current script to the cached one; if they differ, the entry is invalidated and the sub-program is re-parsed and re-compiled (`Program.cs:323-331`). The principle scales: any region of the program that has a stable textual identity admits the same treatment as the whole.

3.2 Interpretation retained

The runtime preserves interpretation as a first-class execution mode, not as a legacy fallback. When a script presents itself without user parameters, or when a per-actor policy directs the runtime to interpret regardless of parameter presence, the program executes by walking the AST directly: each Statement carries an `Execute` method that runs against the program's current state, with no expression tree built and no delegate compiled. The interpreted path costs zero compilation but pays the cost of tree traversal at every invocation. Its place in the runtime is precisely the inverse of the compiled path's: useful where invocation is one-shot or unanticipated, useless where the program will be reused.

The decision is governed by a per-actor `CompilationModePolicy` enum with three values — `Automatic`, `AlwaysCompiled`, and `AlwaysInterpreted` — declared on the actor and defaulted to `Automatic` (`Actor.cs:8-13`). On the parametric path, `PrepareCommandProgram` calls `AdjustCompilationMode` with `useInterpretedMode: false`; on the literal path, with `useInterpretedMode: true` (`ActorHandler.cs:1117-1131`). Under `Automatic`, the policy honors that hint: parametric programs compile, literal ones interpret. Under `AlwaysCompiled` or `AlwaysInterpreted`, the hint is overridden in favor of the policy's name (`Program.cs:134-150`). Execution itself is dispatched by another switch on the same policy: `Perform` calls `ExecuteExpression` if the program is in compiled mode and `Execute` otherwise (`ActorHandler.cs:1955-1968`).

Caching applies asymmetrically to the two principal kinds of DSL invocation. For commands — programs that mutate actor state and persist to the journal — the runtime caches only when the script declares user parameters (`ActorHandler.cs:1117`). For queries, checks, and emit invocations — programs that read state without persistence — the runtime caches when the script declares user parameters or when no parameters are supplied at all (`ActorHandler.cs:1405`). The asymmetry is operational. Commands run under a write lock and serialize, so caching pays only when the same parametric program is invoked many times. Queries run under a read lock and parallelize, so a cache entry amortizes regardless of parametric reuse — a frequently-emitted query gains from caching even when its parameter set is fixed. One final observation: cosmetic variation in the script's text across releases generates distinct cache identifiers, but the runtime's reaction mechanism — treated in a companion paper — matches behavior against semantic patterns rather than identifier equality, so coherence is preserved across textual variants.

3.3 Hot-loaded DSL programs over a stably-loaded domain

The third element of realization is the runtime's support for hot-loaded DSL programs against a stably-loaded domain library. The phrase deserves care, because it is easy to misread. The hot element is the DSL: at any time during

the actor's life, a new script — never seen before by this runtime instance — can be supplied, parsed, prepared, and executed without restart, redeployment, or reflection over a re-loaded assembly. The stable element is the domain library: the host-language types and methods that the DSL invokes are loaded once, by reflection over the assemblies configured as the actor's domain libraries, and held in a cache for the lifetime of the process. Hot-loaded programs combine stable domain elements in new ways; they do not bring new domain elements into being. The runtime is hot at one layer and stable at the layer beneath it.

The mechanism is brief in the source. When an actor is constructed, the public types of its configured library assemblies are walked once by reflection and cached against a deduplicated key: `DomainLibraries.GetOrLoad(params Assembly[])` (`DomainLibraries.cs:77-115`, with both a single-assembly and a multi-assembly overload), invoked from `ActorHandler` (`ActorHandler.cs:61`) with the actor's `LibraryAssemblies`. The cache survives the lifetime of the process; the same assembly set, loaded by a second actor, reuses the same domain dictionary. Against that fixed surface, new DSL scripts arrive through the actor's fluent interface — `ActorV2.Using(scriptForChk, scriptForCmd)` (`ActorV2.cs:32`) — at any time the actor is running. There is no AppDomain reload, no `AssemblyLoadContext` unload, no file-watcher over the assembly: the surface area is fixed at first load and dynamics live entirely above it.

The operational consequence is direct: a running actor can be queried or commanded with a script formed at call time, against any combination of the domain operations the configured library assemblies carry. A configuration adjustment, a one-off audit query, a derivation that the published endpoints do not anticipate — each can be expressed as a fresh DSL program and executed against live actor state without intermediate deployment. The relationship resembles gRPC in its directness — a procedure call against the live system rather than a navigation of REST resources — but without a pre-declared interface contract: the script itself is the contract, formed at call time. What has been shown in this section is that separability is not merely a formal property of DSL programs but one that a concrete runtime can recognize, act upon, and encode into its execution, caching, and persistence behavior. The realization characterized here is one of an extensible family: other actor runtimes — Akka, Orleans, Erlang/OTP — admit, in principle, the same construction, since a parameterized DSL surface above a host-language domain library would yield the same four-fold coherence under the same separability commitment. The expressive reach of such on-the-fly programs depends on the richness of the verbs the domain library exposes, which the next section takes up.

4. Verb richness: surface vs depth

Separability makes the program nameable; verb richness determines how much domain meaning that name carries. The verbs that a DSL invokes are not method calls on data structures. A verb in this setting names an operation against a live, stateful actor — an orchestration that the host language has been

written to perform on behalf of the domain. A single verb may invoke many internal methods, traverse domain relationships, evaluate derived properties, trigger downstream behaviors, and write to multiple regions of state, all in the course of executing one DSL statement. Where a setter assigns a field, an actor verb concludes a transaction — moving the actor from one consistent state to another. The asymmetry between the surface — the few characters that name the verb in the script — and the depth — the volume of domain computation that runs when it is called — is structural, not incidental.

Consider a verb that any reader from an e-commerce or supply-chain background will recognize: the `Confirm` of a purchase order. The script that names this verb at the DSL surface is short — only a handful of tokens. What runs when the script is invoked is substantially larger: a verification that the order's reservations are still valid, a hold-to-allocation conversion across one or more inventory ledgers, a tax computation that depends on the order's line items and their fiscal contexts, a posting to the financial ledger, and a set of reaction triggers that propagate the confirmation to subscribed views and downstream actors. Each of these steps is itself a tree of host-language method calls; the leaf operations include arithmetic, predicate checks, persistence writes, and outbound messages. The runtime gives the consistency boundary that separates this depth from its surface first-class support: a check script and a command script can be supplied together via `ActorV2.Using(scriptForChk, scriptForCmd)` and dispatched as a single invocation through `PerformCheckThenCommand` (`ActorV2Invocation.cs:69`). The check runs against the actor's current state under a read lock; if its assertions hold, the command runs under a write lock and is persisted to the journal. If the check fails, no state change is applied and no journal entry is written — the actor's consistent state is preserved by construction.

Confirm is one example among many. The same asymmetry obtains for any verb that names a domain transition rather than a field assignment — Settle, Allocate, Reconcile, Release, and a hundred others that a well-designed domain library will expose. Verb richness is not an artifact of this particular runtime; it is a property of how domain operations are conceived and named. The implication folds back into the journal density argument of §2.3: when each persisted entry refers to a domain operation that may run dozens to hundreds of internal method calls, the journal's compactness in bytes is matched by an enormous compactness in semantic information per byte. The journal does not record state changes; it records named transactions, each of which carries its full operational meaning by reference to the domain library that knows how to run it. The value of separability would be modest if the verbs it named were shallow. It becomes profound when each name stands for a full domain transaction.

Under porous substrates, the asymmetry inverts entirely. There, every domain element must persist into a relational schema, and deep domain logic becomes a serialization burden — each derived state, each accumulator update, each new field propagates as schema complexity, ETL transforms, and projection

overhead. Under a code-as-data substrate, the same depth is rewarded, not penalized: the domain library can be made arbitrarily expressive without expanding what the journal must record. Complex abstractions compose without friction: no DTOs, ORM annotations, or projection layers stand between the domain operation and its persistence. Porosity makes deep domains expensive to represent; anti-porosity makes them economical to compose. The empirical magnitude of these numbers — how many operations a typical verb dispatches, how many bytes a typical journal entry occupies, how the ratio behaves at scale — is the subject of §5.

5. Empirical results

5.1 Methodology

The measurements that follow span two complementary axes. The first axis is *program complexity*: a synthetic straight-line arithmetic kernel parametric on integer inputs (depth 5 to 100 statements); a DSL-rich kernel exercising control flow (for, if), arithmetic, and parameter binding (~500 dispatched operations per invocation); and a multi-item purchase command operating against an external rich-domain aggregate. The first two tiers isolate runtime overhead independent of host-language work; the third locates that overhead within a representative end-to-end transaction.

The second axis is *host codebase*. The production-verb measurements are replicated against two independent open-source MIT-licensed DDD aggregates that share a structural shape (rich behavioral methods on aggregate roots) but represent disjoint business domains: `dotnet/eShop`'s `Order` aggregate (e-commerce ordering with multi-item cart and a four-step state machine), and `kgrzybek/modular-monolith-with-ddd`'s `SubscriptionPayment` aggregate (subscription billing with a payment-lifecycle verb). The dual-codebase design is deliberate: if the measured properties of compilation, caching, and journaling are structural consequences of the runtime rather than artifacts of a particular domain, the same property should appear on both codebases. The selection criterion was structural similarity (DDD aggregates with rich behavior methods rather than CRUD entities) over an unrelated business domain. Both aggregates are reachable as public source for reproduction; Appendix A lists the per-section datasets, the runtime branch they were produced from, and the commit SHA.

All measurements use Stopwatch instrumentation around the runtime's compilation and execution paths, with N 1,000 invocations per cell after warm-up runs are discarded; cache and pool hit rates use Interlocked counters; journal-density measurements parse the runtime's binary journal format directly. The bench follows a bootstrap-then-measurement pattern: a non-parametric bootstrap script establishes the facade in the actor's persistent symbol table once, and subsequent timed iterations invoke a parametric measurement script that binds per-iteration values via the runtime's parameter API. The script string

stays constant across iterations, so the compiled-program cache hits on every call — the regime the amortization argument names.

5.2 Compilation amortization

Compiled execution shows a p50 speedup over interpreted execution that varies with where the program's work resides. For a synthetic arithmetic kernel of depth 100, the speedup is 4.10× (N=1,000 invocations per cell). For a DSL-rich kernel of ~500 dispatched operations exercising for-loops, conditionals, and parameter binding, the speedup is 2.92×. For a multi-item purchase verb against the eShop `Order` aggregate — one DSL dispatch cascading through `Order` ctor, four `AddOrderItem` calls, and a four-step state-machine walk to Shipped — the speedup is 1.49× (interpreted 5.5 µs p50, compiled 3.7 µs p50, N=1,000). The pattern is monotonic: the more of the per-invocation work is DSL-bound, the larger the speedup; the more it is domain-bound, the smaller. The runtime amortizes only the AST traversal overhead — the host-language code dispatched by the verb runs identically in both modes.

Specialization is paid for at first encounter, not at every invocation. For straight-line arithmetic kernels, the cost of compiling a single program ranges from 0.8 ms (p50, scripts of 5-29 statements) to 4.1 ms (p50, scripts of 80-104 statements); for richer DSL kernels exercising control flow, from 1.1 ms to 5.3 ms over the same depth range. The cost grows linearly in statement count — approximately 44 µs per statement for arithmetic, 43 µs per syntactic structure for richer kernels. For the eShop purchase verb, the cold compile cost is 281 µs at p50 (394 µs at p95, N=100 distinct script variants forcing cache misses). Against the 1.8 µs per-invocation speedup over interpreted execution for the same verb, this cost recovers itself after approximately 156 invocations. Any actor whose lifetime exceeds the first second of operation amortizes its compilation investment in full. These numbers are not performance claims but empirical confirmation of the structural amortization predicted by separability.

5.3 Cache amortization

The same amortization pattern applies recursively to sub-programs. A parameter declared with the Eval modifier carries its own sub-script that the runtime treats as a separable program in miniature; on a cache hit, the sub-program pays only the cost of invoking its cached delegate, while on a cache miss — when the sub-script's text differs from the cached entry — it re-parses, rebuilds, and re-compiles. Across three sub-program complexities (a two-term arithmetic expression, a fifty-term arithmetic kernel, and a method call against a facade-bound counter on the eShop side), the cache-hit cost ranges from 1.7 to 2.2 µs at p50, while the cache-miss cost ranges from 223 to 263 µs at p50. The ratio between cache miss and cache hit sits at approximately 120× regardless of sub-program complexity — a two-orders-of-magnitude separation that confirms the principle extends to any region of the program with a stable textual identity.

Allocation pressure is amortized at a finer granularity by parser and parameter pools. Three workloads measured under AlwaysCompiled policy: 1,000 single-thread invocations against a stable parametric script recorded a 100% parameter-pool hit rate; 1,000 single-thread invocations against distinct cold-cache scripts recorded a 100% hit rate on both pools; 5,000 invocations across 8 parallel threads against distinct scripts recorded 99.90% on parsers and 99.86% on parameters — twelve total misses across the parallel workload's ~10,000 pool rents. Allocation of new Parser and Parameters instances is incurred at startup and under brief thread-fanout transients only; under steady state, both pools service the rent without allocation.

5.4 Journal density

In a parametric purchase workload against the eShop `Order` aggregate — a nine-line DSL script cascading through order construction, four `AddOrderItem` calls, and a four-step state-machine walk — 99.8% of journal entries land as compact action references: 1,001 invocations produce 1,001 action entries plus a single Define entry that carries the script body once. Each action entry's payload averages 115 bytes — the argument vector for seventeen parameters (user identity plus four products' details plus shared modifiers). The Define entry's payload is 642 bytes — the script body itself, persisted once. Had each invocation stored the literal script text instead, the action payload would be 5.6× larger. The runtime's choice to persist named operations rather than their textual content is what makes this density possible. This density does not arise from compression techniques but from reference instead of duplication.

The same compaction structure appears on the second host. Against Grzybek's `SubscriptionPayment` — a two-statement DSL surface (`NewWaitingPayment` followed by `MarkAsPaid`) over a payment-lifecycle aggregate — N=1,000 parametric invocations produce 1,001 Action entries (99.8% of the journal) plus a single Define entry of 207 bytes carrying the script body. Action payloads average 60 bytes. The literal-script storage would be 3.5× larger. The ratio is more modest than eShop's because both the script body and the argument vector are smaller on a narrower verb; the structural property — arguments scaling with invocations while the body is persisted once — is identical.

The magnitude of the ratio is a function of how the host's domain API surfaces its data, not of the compaction mechanism. eShop's `AddOrderItem` carries the full product specification per item — pid, name, price, discount, picUrl, units — so per-iteration argument vectors are dense (115 B). Grzybek's `NewWaitingPayment` accepts five primitive parameters and `MarkAsPaid` takes none, so the argument vector is short (60 B). Hosts whose verbs identify catalog entries by short stable references compound the ratio further; hosts whose verbs string together longer parametric sequences also compound it. Either way the structural property holds: arguments scale with invocations; the script body does not.

5.5 Verb richness

For the eShop purchase verb the DSL script dispatches 9 host-language invocations per invocation, exactly reproducible across 1,000 measured runs. Static forward closure of the call graph from those 9 entry points reaches 24 methods within `dotnet-eShop/src/Ordering.Domain` — measured by a syntactic walker that traces method-to-method edges from the entry points, excludes trivial accessors, and treats the project boundary as a structural ceiling. For the Grzybek `SubscriptionPayment` verb the dispatch count is 2 (the facade absorbs the value-object assembly into one host-language call from the DSL's perspective) and the closure reaches 73 methods within the project's `Payments.Domain` plus its shared `BuildingBlocks/Domain` (275 declarations indexed, 105 trivial accessors filtered). The / asymmetries are 2.7× for eShop and ~36× for Grzybek.

These two numbers do not represent the ceiling of the mechanism — they represent its floor. Both `Ordering.Domain` and `Payments.Domain` are RDBMS-anchored DDD aggregates: their authors designed against the anticipation of EF Core hydration, and that anticipation flattens polymorphism, caps graph depth, and externalizes references as foreign keys. The marks are directly visible in the source — `private set` on every property, explicit `EF Core 2.0 owned entity` annotations on value objects, `int?` foreign keys instead of object references, and a deliberate absence of polymorphism on the aggregate roots. Domains developed without this pressure on their representation grow polymorphism, deeper graphs, and richer cross-class behavior; the same mechanism then traverses substantially further. The / magnitude is therefore a measurement of how much the host's persistence anchor compresses the domain — not of the mechanism the runtime applies once a verb is invoked. The mechanism is what §4 anticipates: a verb whose surface signature names an orchestration deeper than its syntax suggests. The depth of that orchestration is a property of the host's domain, not of the runtime that dispatches into it.

6. Counter-arguments

6.1 *"This is just JIT compilation"*

A reviewer familiar with managed runtimes might object that the runtime described here is just-in-time compilation in disguise. The objection conflates two distinct compilation events. The .NET CLR's JIT translates IL bytecode into native machine code at first invocation of any method; this happens identically whether the runtime executes its DSL by walking an AST or by invoking a compiled delegate. What the runtime adds is a separate, prior compilation: from DSL script to typed Expression tree to IL — the work that produces the delegate the JIT will eventually translate. The speedup measured in §5.2 is a function of this DSL→IL stage, not of the IL→native stage. Both modes deliver IL to the JIT in the same way, so JIT effects cancel; what remains is the AST traversal overhead that the DSL→IL stage amortizes away. Calling this "just

JIT" would erase the layer of specialization that separability makes possible.

6.2 *"Why retain interpretation? Just compile everything"*

A second objection: if compilation is so much faster, why retain interpretation at all? The answer is that compilation is economical only when the program will be reused. A script seen once and never again — an ad-hoc query against a live actor, a one-off configuration adjustment, an exploratory invocation formed at call time — provides no future invocations against which to amortize the compilation cost. Forcing such scripts through the compilation pipeline pays the 1.4 ms cold cost (§5.2) and discards the result; the compiled delegate is never invoked a second time, and the journal records the script literally rather than as a referenced action. The runtime's `AlwaysCompiled` policy permits this on demand, but the default policy correctly recognizes that not every script is amortizable. Separability is necessary for compilation to make sense; reuse is the condition under which compilation is economical.

6.3 *"Dual paths add memory cost"*

A third objection: maintaining a compiled delegate per parametric program inflates memory at scale, and the dual-path arrangement compounds the cost. The measurement disagrees. In a single-actor cache holding 100, 1,000, 10,000, and 100,000 distinct parametric programs under `AlwaysCompiled` policy, the per-entry footprint stabilizes at approximately 6 KB across all four scales: 6,039 bytes per entry at 100 programs, 5,981 at 1,000, 5,977 at 10,000, and 5,930 at 100,000 — no super-linear overhead, no hash-table degradation. The marginal memory of an invocation against an already-cached program is effectively zero: 100,000 invocations against a single cached program retain 104 bytes total. A production actor caching dozens of distinct parametric programs and invoking them at scale incurs a memory cost on the order of hundreds of kilobytes. The cache scales linearly with distinct programs, not with invocations — well below the threshold at which the dual-path arrangement would be a structural concern.

6.4 *"This is just SQL prepared statements"*

A final objection: this is just SQL prepared statements rebranded — the same parametric-template-plus-bind-values pattern that relational databases have used for decades. The pattern is indeed the same; what differs is what the runtime persists. A SQL prepared statement is parameter-separable and admits compilation and caching: the database parses it once, builds a query plan once, and reuses both for many invocations with different bind values. But what the database persists is row data — the projections produced by executing the statement against the underlying tables. The parameterized statement itself is not the persisted artifact; the rows it touches are. The runtime described here persists the operation rather than its row-level effects. Two of the four faces of separability — compilation and caching — are present in both systems; the journal density that follows from persisting the named operation is what SQL

prepared statements do not achieve. Separability is necessary; pairing it with code-as-data is what produces the dense journal.

6.5 *"A verb dispatching dozens or hundreds of host methods produces an opaque runtime"*

A reviewer might object that a verb whose static call-graph closure reaches dozens or hundreds of host-language methods (§5.5) produces an opaque runtime, and that debugging across such depth is intractable. The objection misreads the architecture. The depth lives in the host language: domain classes, methods, business rules — debuggable with the standard tools the host already provides. The DSL surface only names what the host invokes; it does not introduce a separate execution layer that the host debugger fails to penetrate. Standard practice in the runtime described here proceeds by test-driven development with end-to-end test cases, with breakpoints and step-through performed in the host language; once a script is moved to a runtime endpoint, the same breakpoints continue to apply against the domain library it invokes. More consequentially, the journal makes post-mortem debugging tractable in a way porous substrates do not allow. When a production failure surfaces at journal entry id N, that entry refers — by name — to the exact script that produced the defect, parameterized by the exact values the script consumed. A breakpoint at the journal entry, a step-through of the named operation, and the defect reproduces; the failure is then replicated in a small end-to-end test case, fixed, and released. The dense journal is not the opacity it might appear to be on a superficial reading; it is among the runtime's most powerful debugging artifacts, because the persisted artifact is the operation that ran, not a downstream trace of its effects.

Each objection treats compilation, caching, and journaling as independent engineering choices. The argument of this paper is that they are consequences of a single structural property; the objections dissolve when that property is made explicit.

7. Related work

7.1 Partial evaluation

The structural condition this paper formalizes has a close ancestor in **partial evaluation**, as developed by Jones, Gomard, and Sestoft (1993) and the Futamura projections (Futamura 1971, 1999). Partial evaluation specializes a program with respect to a subset of inputs known in advance, producing a residual program that depends only on the dynamic arguments — and it requires, structurally, that the program's static and dynamic inputs be separable. Where the partial-evaluation literature applies the technique to general-purpose host languages, this paper applies the same separability principle to a domain-specific language whose programs are short, parametric by construction, and stored as journal entries; in this setting, separability is syntactic, decidable from the pro-

gram's surface rather than from static analysis. The further consequences traced in §2 — caching and dense journaling — are not part of the partial-evaluation tradition; they follow from pairing separability with code-as-data persistence.

7.2 Lambda lifting

Lambda lifting (Johnsson 1985) is a program transformation that rewrites locally-defined functions whose free variables capture an enclosing scope into top-level functions that receive those variables as explicit parameters. The transformation makes the function's dependencies syntactically visible — and, by doing so, makes the function amenable to ahead-of-time compilation, separate compilation, and uncluttered call-graph analysis. The structural insight is the one this paper generalizes: dependencies that remain implicit in the program's body cannot be specialized over, but the same dependencies promoted to the surface can. Where lambda lifting transforms programs that were already written and externalizes their captured environment, the runtime described here requires programs to declare their values as externalized parameters from the start. Lambda lifting opens compilation; this paper extends the same principle to caching, dense journaling, and replication-bounded entropy by joining separability with code-as-data persistence.

7.3 Closure conversion

Closure conversion (Appel 1992) is a compilation technique that translates a function with captured lexical environment into an explicit closure record — a pair of function pointer and environment data — that the runtime carries as a single value. Closure conversion makes implicit lexical dependencies explicit, but preserves them as runtime data: the closure travels with its environment record, retained in memory, and re-bound on each invocation. Lambda lifting and the principle of this paper sit on the other side of the same choice: rather than carry the environment, eliminate it by promoting the variables that would have been captured into top-level parameters supplied at the call site. The runtime described here makes that choice mandatory and declarative — values must be supplied at invocation, not captured from a surrounding scope — which is what permits the journal to record the program by name without any captured-environment record traveling alongside it.

7.4 Template instantiation

Template instantiation in general-purpose languages — exemplified by C++ templates (Stroustrup 1986–present; Vandevoorde and Josuttis 2017) and the broader tradition of generic programming — applies the separability principle in another setting: a template body names operations parameterized by types or compile-time constants, and the compiler instantiates each template with concrete parameters at compile time, producing specialized code per instantiation. The structural pattern is the same as the one described here: a program that admits parametric reuse must be written so its parameters are separable from

its body. Template instantiation operates over types and compile-time values; the runtime described here applies the same principle over runtime values supplied at invocation. The distinction is operational: instead of monomorphizing the program once per parameter tuple, the runtime compiles the program once and rebinds its arguments on every call. Both arrangements presuppose what this paper formalizes.

7.5 Prepared statements

Prepared statements in relational database systems (SQL standard ISO/IEC 9075; Hellerstein and Stonebraker 2007) are the closest operational analog to the runtime described here. A prepared statement carries a parametric template (with placeholders for bind values), the database parses and plans it once, caches the plan, and reuses both for many invocations with different argument vectors — the same parametric-template-plus-bind-values pattern §1.1 used as pedagogical anchor for the principle this paper formalizes. The two systems share separability and its first two consequences: compilation (the query plan) and caching (the plan cache). They diverge on what the runtime persists. A relational database persists row data; the prepared statement itself is not the persisted artifact. The runtime described here persists the named operation rather than its row-level effects, completing the four-fold coherence with journal density and replication-bounded entropy. The principle this paper formalizes generalizes beyond any one runtime — it characterizes the structural condition any DSL runtime must satisfy to admit compilation, caching, and dense persistence at all.

These traditions arise in different domains — compilers, functional languages, generic programming, database engines — but they converge on the same structural observation: programs whose dependencies are syntactically separable from their bodies admit forms of specialization, reuse, and compact representation that programs with embedded values cannot. This paper isolates that observation as a single principle and traces its full runtime consequences in a DSL setting.

8. Conclusion

The argument of this paper can be stated compactly. Program–value separability — the syntactically decidable property that a DSL program declares user parameters rather than embedding values in its body — is the structural precondition under which compilation, caching, and dense journaling become economically meaningful. The four runtime faces traced here are not parallel design choices: they are downstream consequences of separability, paired with code-as-data persistence in the case of dense journaling. The runtime characterized in §3 realizes the principle concretely; the magnitudes reported in §5 confirm the predicted structural amortization across two open-source DDD aggregates — a compiled-versus-interpreted speedup that scales monotonically

with DSL-bound work (1.49× on the eShop purchase verb to 4.10× on a synthetic arithmetic kernel), a cold compile cost (281 µs for the eShop purchase verb) that recovers itself after roughly 156 invocations, and a journal in which 99.8% of parametric entries are compact action references producing a footprint several-fold denser than the equivalent literal-script storage. The magnitudes are floors of the mechanism on RDBMS-anchored DDD aggregates whose authors designed against ORM hydration; the same mechanism applied to host domains without that representational pressure compounds further.

The principle takes its place alongside the analysis of the prior paper of this series, *Anti-porous Architecture*, which characterized porosity as a representational sparsity problem and density preservation as the operational consequence of an event-sourced DSL journal. Where the prior paper named the defect that domain representations on tabular substrates accumulate, this paper names the condition under which that defect can be avoided.

Prior paper	This paper
Problem: porosity	Condition for avoiding it
Density preserved	Program–value separability
Homoiconic journal	Stable program identifier
Operations vs state	Functions vs instances

The two papers describe the same phenomenon from opposite sides: density preservation is what is achieved; separability is what makes it achievable. Together, they argue that density in domain representation is not an optimization but a structural property that follows from how programs relate to their values.

The principle is not bound to the runtime characterized in §3. In an Orleans- or Akka-style actor framework with a parameterized command DSL above a host-language domain library, the same precondition would predict the same four-fold coherence; in a service that uses prepared statements over an event-sourced log of named operations, three of the four faces are already available, and the fourth — replication-bounded entropy — follows once the persisted artifact is the operation rather than its row-level effects. The realization in §3 is one example of a construction the principle admits, not the only path to it.

Two extensions of the principle remain to be developed in subsequent work. The fourfold coherence described here does not exhaust the operational consequences of an externalized-parameter, locality-bound substrate: a persistent local write buffer with asynchronous remote replication, for instance, is structurally enabled by the same commitments — the actor's isolation guarantees that locally-buffered, not-yet-replicated entries are invisible to other contexts by construction, not by convention. The persistent buffer and its zero-downtime implications are treated in a companion paper. A second companion paper takes up the reaction mechanism whose semantic-pattern-matching has been mentioned in passing throughout this argument, and the consistency contract

under which observable state remains coherent across actors. In each case the same structural observation continues to apply — what makes the journal compact, what makes deep domains compose without friction, is the separation of the program from its values. Porosity makes deep domains expensive to represent; anti-porosity makes them economical to compose.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Appel, A. W. (1992). *Compiling with continuations*. Cambridge University Press.

Futamura, Y. (1999). Partial evaluation of computation process — an approach to a compiler-compiler. *Higher-Order and Symbolic Computation*, *12*(4), 381–391. (Original work published 1971 in *Systems, Computers, Controls*, *2*(5), 45–50.)

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, *1*(2), 141–259.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

International Organization for Standardization. (2023). *ISO/IEC 9075:2023 Information technology — Database languages — SQL.*

Johnsson, T. (1985). Lambda lifting: Transforming programs to recursive equations. In J.-P. Jouannaud (Ed.), *Functional programming languages and computer architecture* (pp. 190–203). Springer-Verlag. (Lecture Notes in Computer Science, Vol. 201)

Jones, N. D., Gomard, C. K., & Sestoft, P. (1993). *Partial evaluation and automatic program generation*. Prentice Hall.

Rivera, A. (2026). *Anti-porous architecture: A unified design principle for CQRS + Actor + Event-Sourcing systems* [Paper 1 of this series]. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/01-anti-porosity.md

Stroustrup, B. (1994). *The design and evolution of C++*. Addison-Wesley.

Vandevoorde, D., Josuttis, N. M., & Gregor, D. (2017). *C++ templates: The complete guide* (2nd ed.). Addison-Wesley.

Appendix A — Code references

The references below cite source locations in the Puppeteer codebase. Citations are given as `file:line` pairs against the runtime version current at the time of measurement (see §5.1 for measurement methodology). The codebase currently lives in a private repository; a public-mirror URL will be added to this appendix when the codebase is sanitized for public release. Datasets cited in §5 are stored in the companion `data/` directory of this paper repository, with the same publication caveat.

§1.2 — Formal characterization

Reference	Location	What it shows
Runtime self-documentation as `F(x1,...,xn)`	`ActorHandler.cs:1085`	Comment articulating the parametric program model

Reference	Location	What it shows
Parametric/literal regime documented in code	`ActorHandler.cs:1091-1093`	Comment encoding the binary taxonomy: scripts without user parameters are interpreted, uncached, persisted as Script entries; scripts with user parameters are compiled, cached with an ActionId, persisted as Action entries

§3.1 — Compilation pipeline

Reference	Location	What it shows
Orchestration on first encounter	`ActorHandler.cs:1097-1148`	`PrepareCommandProgram` rents a parser, parses the script, decides parametric vs literal path
AST traversal	`Program.cs:182-244`	`ProgramExpression` walks `Statements`, accumulates `Expression` nodes into a `BlockExpression`
Lambda composition	`Program.cs:244`	`Expression.Lambda<Func<Parameters, Output, string>>` wraps the `BlockExpression`
Compilation step	`Program.cs:163`	`_executable = programExpression.Compile()`
Eval parameter cache structure	`Program.cs:305`	Dictionary keyed by parameter name to `(EvalScript, Compiled)` tuple
Eval recompile on script change	`Program.cs:323-331`	Cache lookup, invalidation, re-parse, re-compile

§3.2 — Interpretation retained

Reference	Location	What it shows
`CompilationModePolicy` enum	`Actor.cs:8-13`	Three policy values (`Automatic`, `AlwaysCompiled`, `AlwaysInterpreted`); default is `Automatic`
Policy hint passed at call site	`ActorHandler.cs:1117-1181`	`AdjustCompilationMode(useInterpretedMode, policy)` — `useInterpretedMode: true` for literal path, `false` for parametric
Policy resolution switch	`Program.cs:134-150`	Sets `IsCompiledMode` from policy + hint
Execution dispatch	`ActorHandler.cs:1955-1968`	`Perform` calls `ExecuteExpression` if compiled, `Execute` otherwise
Commands caching rule	`ActorHandler.cs:1117`	Caches only when `parameters.HasUserParameter()` is true
Queries caching rule	`ActorHandler.cs:1405`	Caches when `EMPTY_PARAMETERS` *or* `HasUserParameter()` is true (asymmetry vs commands)

§3.3 — Hot-loaded DSL programs

Reference	Location	What it shows
Domain library loading	`DomainLibraries.cs:77-165`	`GetOrLoad(params Assembly[])` reflects public types and caches them once per deduplicated assembly set; a single-assembly overload remains for the back-compat path
Library binding to actor	`ActorHandler.cs:61`	`libraries = DomainLibraries.GetOrLoad(LibraryAssemblies)`

Reference	Location	What it shows
Fluent script invocation	`ActorV2.cs:32`	`Using(scriptForChk, scriptForCmd)` introduces new scripts at runtime against the cached domain

§4 — Verb richness

Reference	Location	What it shows
Check-then-command fluent dispatch	`ActorV2Invocation.cs:69`	`PerformCheckThenCommand()` — read-lock check + write-lock command + journal write, with rollback on check failure

§5 — Empirical datasets

The datasets that produced the magnitudes reported in §5 are stored in the companion `data/` directory of the paper repository. The production-verb measurements are produced against two independent open-source aggregates:

- **eShop** — `dotnet/eShop` (MIT), `src/Ordering.Domain/AggregatesModel/OrderAggregate/Order.cs`. Multi-item purchase via 10-arg constructor plus four `AddOrderItem` calls plus a four-step state-machine walk to Shipped.
- **Grzybek** — `kgrzybek/modular-monolith-with-ddd` (MIT), `src/Modules/Payments/Domain/Subscri` Subscription payment lifecycle via `Buy` factory plus `MarkAsPaid`.

The synthetic kernels in §5.2 (arithmetic, DSL-rich) carry no external-codebase dependency.

Section	Dataset directory	Lab	Host	What it contains
§5.2 (compilation speedup, production verb)	`data/lab01-eshop/`	Lab 1 Run 3	eShop	Compiled vs interpreted bench, N=1000, bootstrap-then-measurement

Section	Dataset directory	Lab	Host	What it contains
§5.2 (compile cold cost, production verb)	`data/lab02-eshop/`	Lab 2 Tier 3	eShop	Cold compile cost per distinct script variant, N=100
§5.3 (eval cache hit vs miss)	`data/lab03-eshop/`	Lab 3 Tier C	eShop	Stable vs mutating eval text; per-iteration end-to-end and isolated eval-compile ticks
§5.4 (journal density)	`data/lab04-eshop/`	Lab 4	eShop	Action / Script / Define entry counts and payload bytes parsed directly from BinaryEvent-Codec jour-nal_*.bin
§5.4 (journal density, replication)	`data/lab04-grzybek/`	Lab 4	Grzybek	Same entry-type analysis applied to the Subscrip-tionPayment lifecycle verb
§5.5 (verb richness)	`data/lab05-eshop/`	Lab 5	eShop	DSL dispatch counter (runtime) plus Roslyn forward closure over `Ordering.Domain`

Section	Dataset directory	Lab	Host	What it contains
§5.5 (verb richness, replication)	`data/lab05-grzybek/`	Lab 5	Grzybek	+ over `Payments.Domain` plus shared `BuildingBlocks/Domain`

Each dataset directory contains a `headline.md` summary, raw CSVs, and a Git SHA stamping the runtime version against which the lab was run. The Roslyn walker source for the half of §5.5 lives at `labs/lab05-eshop-roslyn/` and `labs/lab05-grzybek-roslyn/` — both are self-contained .NET 9 console projects that can be re-executed against the public source trees of the respective aggregates.